[Bigdata Analytics]

linear Regression

DaeHan Ahn

AI Convergence

울산대학교
UNIVERSITY OF ULSAN

Regression Analysis

What is Regression ?

- Regression analysis is a statistical method used to model the relationship between **independent variables (features)** and a **dependent variable (target)** in order to predict continuous numerical values.

Purpose of Regression

- Understand how input variables affect the target variable
- Estimate unknown continuous values
- Quantify the strength and direction of relationships
- Support decision-making based on data

Types

- **Simple Regression**: an independent variable → $y = WX + b$
- **Multiple Regression**: more than 2 independent variables → $y = W_1X_1 + W_2X_2 + \cdots + b$
- **Logistic Regression**: When the dependent variable is categorical (0 or 1)

Use It When...

- **1. The target variable is continuous**
  - How much will next year's revenue be?
  - What will this patient's blood pressure level be?
- **2. You want to predict numerical values**
  - If we increase ad spend by $10K, how much will sales grow?
  - If a student studies 1 more hour, how many more points will they score?
- **3. You Want to Identify the Most Influential Variables**
  - What drives house prices more — size or location?
- **4. Your outcome is a continuous number** -> Standard Regression
- **5. Your outcome is a category (Y/N, P/F)** -> Logistic Regression

Limitations of Regression Analysis

- **1. Assumes a Linear Relationship**
  - If the real relationship is curved or complex, the model will be inaccurate
- **2. Sensitive to Outliers**
  - A few extreme data points can heavily distort the entire model
  - One bad data point can pull the regression line significantly off course
- **3. Correlation ≠ Causation**
  - Just because two variables are related doesn't mean one causes the other
  - e.g., Ice cream sales and drowning rates are correlated — but ice cream doesn't cause drowning (both increase in summer)
- **4. Multicollinearity (Multiple Regression)**
  - When independent variables are too correlated with each other, coefficients become unreliable
- **5. Requires Clean, Sufficient Data**
  - Missing values, small sample sizes, or biased data lead to poor results
  - Garbage in → Garbage out

Basic Assumptions of Linear Regression

- **1. Linearity**
  - The relationship between independent variables and the dependent variable should be approximately linear
  - ***Check with:*** Scatter plot, Residual vs Fitted plot
  - ***Solution:*** apply log/polynomial transformation
- **2. Homoscedasticity (Constant Variance)**
  - The variance of residuals (errors) should be consistent across all levels of X
  - ***Check with:*** Residual plot, Scatter plot
  - ***Solution:*** use weighted regression
- **3. Normality of Residuals**
  - The residuals (errors) should be normally distributed
  - ***Check with:*** Histogram, Q-Q plot, Shapiro-Wilk test
  - ***Solution:*** increase sample size
- **4. No Multicollinearity** *(Multiple Regression only)*
  - Independent variables should not be highly correlated with each other
  - High correlation makes coefficients unstable and uninterpretable
  - ***Check with:*** *VIF (Variance Inflation Factor)*
  - ***Solution:*** *remove or combine correlated variables*

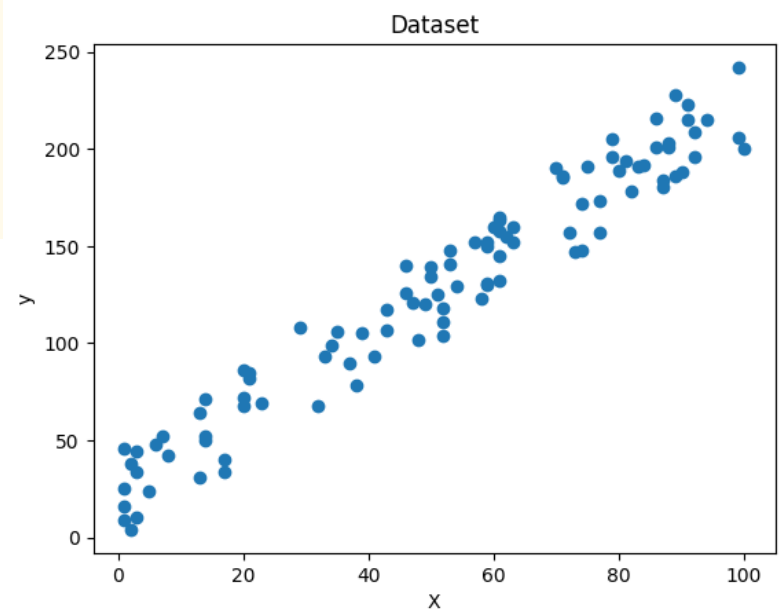
Evaluation Metrics for Linear Regression

- **1. Mean Absolute Error (MAE)**
  - Measures the average absolute difference between actual and predicted values
  - MAE = $\frac{1}{n}\sum|y_i - \hat{y}_i|$
  - Easy to interpret
  - Same unit as target variable
- **2. Mean Squared Error (MSE)**
  - Squares errors before averaging
  - MSE = $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$
  - Penalizes large errors strongly
  - Sensitive to outliers
- **3. Root Mean Squared Error (RMSE)**
  - Square root of MSE
  - RMSE = = $\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$
  - Same unit as target variable
  - Widely used in practice
- **4. R-squared (Coefficient of Determination)**
  - Measures how much variance is explained by the model
  - $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$
  - Value closer to 1 means better fit
  - Easy to compare models

Practice: Linear Regression (Scikit-learn)

- Generate Dataset

```
import numpy as np
import matplotlib.pyplot as plt


# Generate Dataset
np.random.seed(42)
X = np.random.randint(0, 101, size=(100, 1))  # Independent Variable (0~100 value, #100)
y = 2 * X + np.random.randint(0, 51, size=(100, 1))  # Target variable (add noise)


# Visualization
plt.scatter(X, y)
plt.title('Dataset')
plt.xlabel('X')
plt.ylabel('y')
plt.show()
```



Practice: Linear Regression (Scikit-learn)

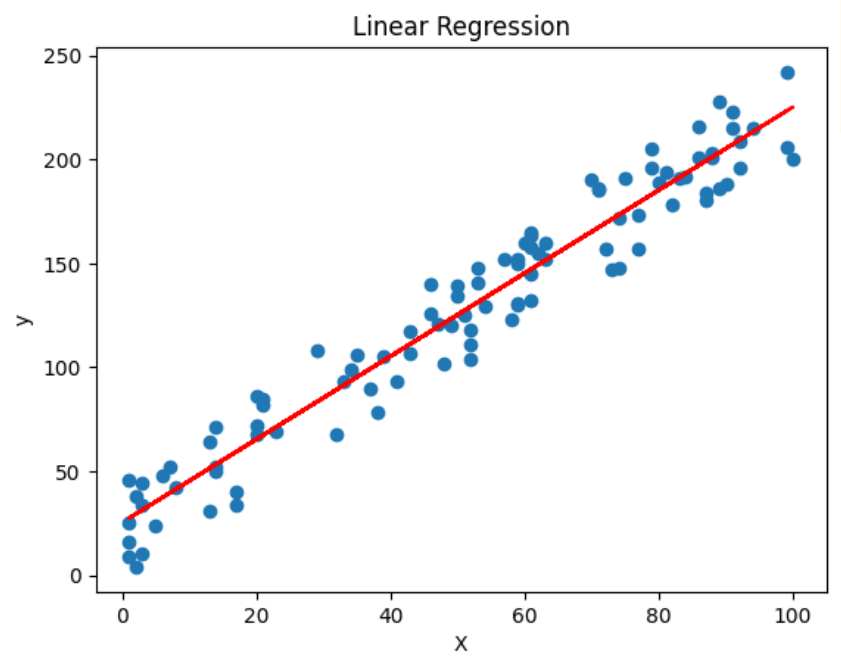
- **Train Regression Model**
  - **Q1: Import LinearRegression from Sklearn library**
  - **Q2: Train the model using 'fit' function**

```
from sklearn.linear_model import __________

model = _______________()
model.___(X, y)
```

Practice: Linear Regression (Scikit-learn)

- **Train Regression Model**
  - **Q: Predict Y for input X, and plot the regression line using plt.plot (matplotlib)**

```
y_pred = model._____(X)

plt.scatter(X, y)
plt.____(X, y_pred, color='red')
plt.title('Linear Regression')
plt.xlabel('X')
plt.ylabel('y')
plt.show()
```



Practice: Linear Regression (Statsmodels)

- **Statsmodels**
  - It basically doesn't have an intercept (bias b)
  - **Q:** Add the intercept using 'add_constant'

```
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt


# Generate Dataset
np.random.seed(42)
X = np.random.randint(0, 101, size=(100, 1)) # Independent Variable (0~100 value, #100)
y = 2 * X + np.random.randint(0, 51, size=(100, 1)) # Target variable (add noise)

# add intercept (bias b)
X_sm = sm._______(X)

# OLS (Ordinary Least Squares) model and train
model = sm.OLS(y, X_sm)
results = model.___()

print(results.summary())
```

Practice: Linear Regression (Statsmodels)

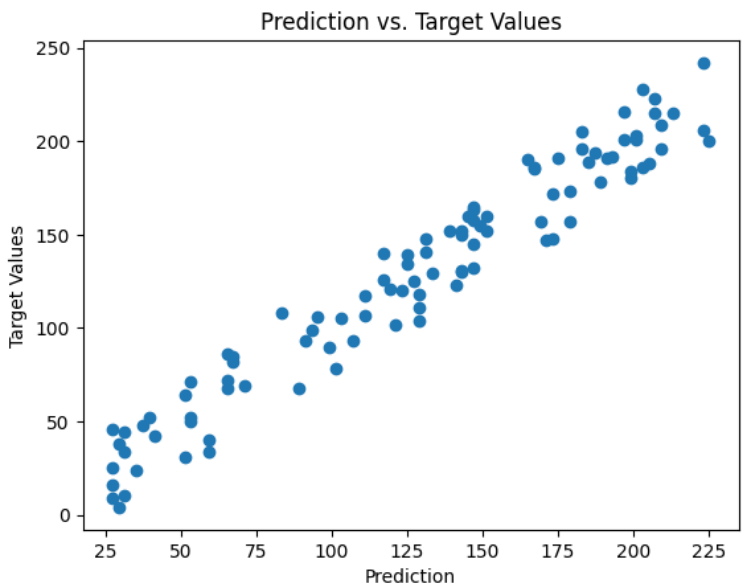
- **Statsmodels (Result)**

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.944
Model:                            OLS   Adj. R-squared:                  0.943
Method:                 Least Squares   F-statistic:                     1649.
Date:                Thu, 19 Mar 2026   Prob (F-statistic):           4.07e-63
Time:                        10:50:08   Log-Likelihood:                -408.60
No. Observations:                 100   AIC:                             821.2
Df Residuals:                      98   BIC:                             826.4
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         25.3585      2.914      8.703      0.000      19.576      31.141
x1             1.9985      0.049     40.613      0.000       1.901       2.096
==============================================================================
Omnibus:                       22.302   Durbin-Watson:                   1.999
Prob(Omnibus):                  0.000   Jarque-Bera (JB):                5.663
Skew:                          -0.190   Prob(JB):                       0.0589
Kurtosis:                       1.898   Cond. No.                         119.
==============================================================================
```

Practice: Linear Regression (Statsmodels)

- **Linearity**

```
pred = results.predict(X_sm)
plt.scatter(pred, y)
plt.xlabel("Prediction")
plt.ylabel("Target Values")
plt.title("Prediction vs. Target Values")
plt.show()
```



If the data points generally form a straight-line pattern, the relationship can be considered linear

If a curved shape or complex pattern appears, it may indicate a nonlinear relationship

Practice: Linear Regression (Statsmodels)

- **Normality of Residuals**
  - **Shapiro-Wilk test**
    - It takes a value between 0 and 1
    - A value closer to 1 indicates that the distribution is close to a normal distribution

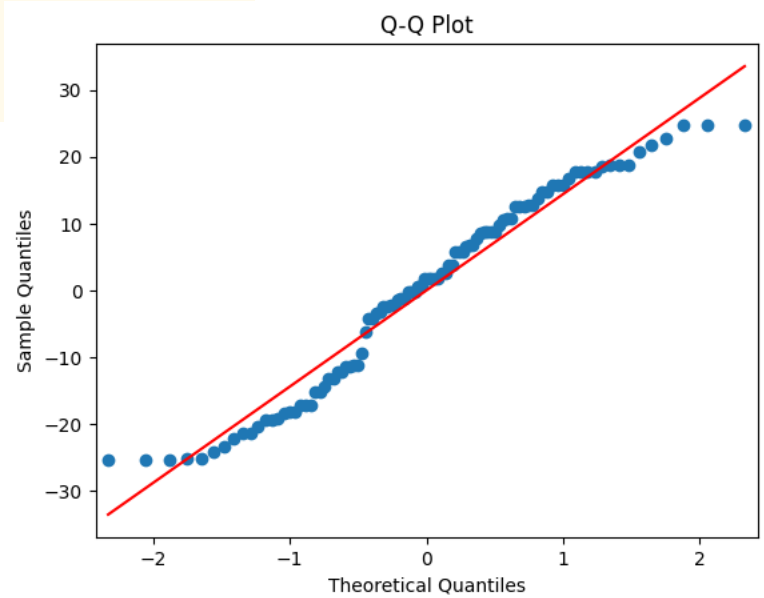
```python
from scipy import stats

# Evaluate residual
residuals = results.resid

# Shapiro-Wilk test
shapiro_test = stats.shapiro(residuals)
print("Shapiro-Wilk Test:\nStatistic:", shapiro_test[0], "\np-Value:", shapiro_test[1])
```

```
Shapiro-Wilk Test:
Statistic: 0.9525925088674045
p-Value: 0.0012341951465015823
```

Practice: Linear Regression (Statsmodels)

- **Normality of Residuals**
  - **Q-Q plot**
    - A QQ plot (Quantile–Quantile Plot) is a graph that compares the quantiles of sample data with the quantiles of a normal distribution
    - If the data follows a normal distribution, the points are generally drawn close to a straight line

```python
# Q-Q Plot
plt.figure(figsize=(6, 4))
sm.qqplot(residuals, line='s')
plt.title('Q-Q Plot')
plt.show()
```



Practice: Linear Regression (Statsmodels)

- **Homoscedasticity**
  - **Breusch–Pagan Test**
    - The **Breusch–Pagan test** is a statistical method used to examine whether the assumption of **homoscedasticity** is valid in a linear regression model
    - The **p-value is more important than the test statistic itself**
      - **p-value ≤ 0.05:** The test is statistically significant, indicating that heteroscedasticity may exist in the residuals
      - **p-value > 0.05:** The test is not statistically significant, suggesting that assuming homoscedasticity of residuals is reasonable

```python
from statsmodels.stats.diagnostic import het_breuschpagan

# Evaluate Residuals
residuals = results.resid

# Breusch-Pagan test
bp_test = het_breuschpagan(residuals, results.model.exog)
print("Breusch-Pagan Test:\nStatistic:", bp_test[0], "\np-Value:", bp_test[1])
```

```
Breusch-Pagan Test:
Statistic: 0.056244060284416086
p-Value: 0.8125339836269723
```

Practice: Linear Regression (Statsmodels)

Multicollinearity

```
import pandas as pd

np.random.seed(42)
X1 = np.random.randint(0, 101, 100)
X2 = X1 + np.random.normal(0, 3, 100)
X3 = np.random.randint(50, 201, 100)
y = 2 * X1 + 4 * X2 + 1 * X3 + np.random.normal(0, 5, 100)

df = pd.DataFrame({'X1': X1, 'X2': X2, 'X3': X3, 'y': y})
df.head()
```

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Evaluate VIF
X = df[['X1', 'X2', 'X3']]
X = sm.add_constant(X)
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print(vif_data)
```

```
  feature         VIF
0   const   12.029618
1      X1  117.709700
2      X2  117.890741
3      X3    1.018220
```

VIF close to 1 generally indicates little or no multicollinearity

- 1<VIF<5: Acceptable level
- 5<VIF<10: Moderate multicollinearity
- VIF>10: variable removal or transformation should be considered

Thank you !

Q & A